

GuidePoint — User Handbook

- [GuidePoint — User & Developer Handbook](#)
 - [1. What GuidePoint Does](#)
 - [2. Who This Guide Is For](#)
 - [3. Repository Layout \(Important Paths\)](#)
 - [4. Prerequisites \(Development / QA\)](#)
 - [5. Get the Source and Run \(Developers\)](#)
 - [6. First Launch — End-User Screens](#)
 - [6.1 Terms & conditions](#)
 - [6.2 QR scanner](#)
 - [6.3 “Say your destination”](#)
 - [6.4 Navigation & instructions](#)
 - [6.5 Instruction replay](#)
 - [7. Obtaining an ATL Demo QR Quickly](#)
 - [8. Alternate Demos Without a Printed QR or Camera Issue](#)
 - [9. Designing Your Own Test QR Payloads](#)
 - [9.1 Inline JSON QR](#)
 - [9.2 Optional start override](#)
 - [9.3 URL-based QR](#)
 - [10. Verification Checklist \(QA Before Demos\)](#)
 - [11. Facilities Partner Notes \(Maps & Hosting\)](#)
 - [12. Troubleshooting Summary](#)
 - [13. From Development Build Toward Stores \(Brief\)](#)
 - [14. Project Team — UT Dallas \(EPICS Computer Science\)](#)
 - [15. License](#)
 - [Appendix A — Canonical Links Inside the Repository](#)

GuidePoint — User & Developer Handbook

Indoor QR navigation with voice-guided routing

Document version	1.1 (May 2026) · pairs with repo release 1.3.4
Repository (web)	https://github.com/nandanpabolu/GuidePoint_NAN
Clone (HTTPS)	https://github.com/nandanpabolu/GuidePoint_NAN.git
Maintainer affiliation	The University of Texas at Dallas · UTDesign EPICS (Computer Science)

1. What GuidePoint Does

GuidePoint helps people navigate indoors when GPS is unreliable:

1. A **QR code** provides a building **floor graph** as structured **JSON** (either embedded directly in the QR or loaded from an **HTTPS** URL printed in the QR).
2. After the map loads, the traveler **speaks a destination** (e.g. “Seminar Hall”). The app matches speech to waypoint **names** in the graph.
3. A* **pathfinding** computes a walking route from a **starting node** to the requested place.

4. The **navigation** screen gives **spoken (TTS) turn-by-turn** hints, optionally uses **step counting** between nodes, and can use optional **TensorFlow Lite** vision models where configured.

There is **no built-in backend server**: maps are ordinary JSON documents. Facilities partners may host maps on **any HTTPS file host** when using URL-based QR payloads.

Full technical overview: root **README.md** in the repository and **docs/ARCHITECTURE_SENSOR_FUSION.md** (engineering deep dive).

2. Who This Guide Is For

Audience	Typical use of this handbook
End users / testers	Install a build someone gives you, grant permissions, scan QR, speak destination → follow cues. Sections 5–9.
Developers & EPICS collaborators	Clone the repo, install Flutter toolchain, run from source, run tests. Sections 3–11.
Facilities partners	Prepare JSON graphs and QR rollouts (docs/MAP_DATA_SCHEMA.md). Sections 10–11.

3. Repository Layout (Important Paths)

After you clone https://github.com/nandanpabolu/GuidePoint_NAN.git, useful locations are:

Path	Contents
flutter_app/	Flutter project — primary app (flutter run from here).
tools/scan_atl_qr.html	Browser page with an ATL sample-map QR for quick testing. Open this file locally in Chrome/Safari/Edge after cloning.
data/maps/ATL_JSON.json	Example building map aligned with docs/MAP_DATA_SCHEMA.md .
docs/	Setup (SETUP_FLUTTER_AND_RUN.md), emulator (ANDROID_EMULATOR_SETUP.md), demos (DEMO_AND_TESTING.md), map schema (MAP_DATA_SCHEMA.md).

4. Prerequisites (Development / QA)

Minimum for **running from source**:

- **Flutter SDK 3.2+** (<https://docs.flutter.dev/get-started/install>)
- **Dart** (bundled with Flutter)
- For **Android**: Android SDK / Android Studio, USB debugging on device or emulator
- For **optional iOS** builds on macOS: Xcode, Apple tooling
- A **physical phone** recommended for microphone, realistic camera QR scanning, and **pedometer** testing (emulators often approximate these poorly)

Always verify toolchain health:

```
flutter doctor -v
```

Accept Android SDK licenses when prompted:

```
flutter doctor --android-licenses
```

First-time setup walkthrough without prior Flutter experience: `docs/SETUP_FLUTTER_AND_RUN.md` in the cloned repository.

5. Get the Source and Run (Developers)

```
git clone https://github.com/nandanpabolu/GuidePoint_NAN.git
cd GuidePoint_NAN
cd flutter_app
flutter pub get
flutter analyze
flutter test
flutter run
```

When **flutter run** lists devices, pick your phone or emulator. On first launch, **accept Camera** and **Microphone** permissions when prompted.

macOS desktop (fast demo, fewer sensor limitations for layout testing):

```
cd flutter_app
flutter run -d macos
```

Deep demo script: `docs/DEMO_AND_TESTING.md` (same repo, docs/ folder).

6. First Launch — End-User Screens

6.1 Terms & conditions

First launch shows **Terms & Conditions**. Acceptance is stored (**SharedPreferences**) so returning users skip this after tapping **Accept**.

6.2 QR scanner

The app opens **camera preview** with **QR decoding**. You must scan a QR whose payload is either:

- Valid **inline JSON** for the navigation graph (compact maps only — very large JSON may exceed practical QR density), OR
- A **literal HTTPS URL string** pointing to JSON (**GET** fetch)

After a successful read, the app parses nodes/edges (`docs/MAP_DATA_SCHEMA.md`) into the pathfinding engine.

6.3 “Say your destination”

Once the graph is loaded, the app listens via **speech-to-text**. Speak one of the **node display names** in the JSON (substring matching is supported). Examples that match the bundled ATL demo: **“Seminar Hall”**, **“Idea Labs”**, **“Hallway Junction”**.

6.4 Navigation & instructions

The **Navigation** screen:

- Gives **spoken** distance/next-step guidance (**TTS**) on an interval cadence
- Tracks **Steps** via **pedometer** on capable hardware
- Optionally uses **camera** + bundled **TensorFlow Lite** assets for landmark confirmation — behavior depends on which **.tflite** files are present under **flutter_app/assets/models/**

6.5 Instruction replay

Use the **list icon** on the navigation app bar (**Stored data** route) to reread textual instructions.

7. Obtaining an ATL Demo QR Quickly

1. Clone or download the repo.
2. On your laptop, open **tools/scan_atl_qr.html** from the repository root in any modern browser — a large QR renders on screen.
3. With **GuidePoint** running on a phone (**QR scanner** screen), scan that QR from the laptop display (brightness up; hold steady).

If microphone permission fails, revisit **Android Settings** → **Apps** → **GuidePoint** → **Permissions**.

8. Alternate Demos Without a Printed QR or Camera Issue

During **development** builds (**debug/profile**):

1. From the scanner screen, tap **“Load sample map”** → loads bundled sample JSON (ATL-style).
2. On the subsequent screen, tap **“Preview navigation UI”** → opens navigation UI with a **fixed sample route**.

These shortcuts are omitted in **release** consumer builds (**flutter build release**). Prefer real QR payloads for stakeholder demos that mimic production behavior.

Guidance without a physical Android device is in **docs/ANDROID_ID_EMULATOR_SETUP.md**; use emulator camera → laptop webcam routing if scanning from the emulator.

9. Designing Your Own Test QR Payloads

9.1 Inline JSON QR

Minimal structure (floors contain **nodes** and **edges**; each node includes **id**, **name**, **position**):

See **docs/DEMO_AND_TESTING.md** for a copy-paste small graph.

9.2 Optional start override

Include root key **start_node_id** when the sticker is glued at a waypoint other than the default graph entry:

```
{
  "building": { "name": "MyBuilding", "floors": [ ... ] },
  "start_node_id": "junction_1"
}
```

9.3 URL-based QR

If the QR string is `https://example.com/maps/site.json` the app **GETs** JSON and parses it exactly like inline content. Use **HTTPS**.

10. Verification Checklist (QA Before Demos)

#	Step	Expected result
1	flutter analyze inside flutter_app/	Reports No issues .
2	flutter test	All tests green.
3	Install debug build (flutter run)	App launches Home → QR flow.
4	Scan scan_atl_qr.html	Map loads → STT activates.
5	Say “ Seminar Hall ”	Route exists → navigation cues play.
6	Deny mic deliberately	Recover by enabling permission + retry speech.

Extended matrix: `docs/DEMO_AND_TESTING.md` §5–6.

11. Facilities Partner Notes (Maps & Hosting)

Author JSON using `docs/MAP_DATA_SCHEMA.md` as canonical structure. Maintain:

- Stable **id** values for nodes (programmatic references) vs human **name** strings (speech targets).
- **edges** with realistic **distance** units (consistent with cues).
- Printed QR placement near graph entry (**start_node_id** when appropriate).

Hosting only requires static file distribution or CDN — no database prerequisite.

Sample graph file: `data/maps/ATL_JSON.json`.

12. Troubleshooting Summary

Symptom	Things to verify
QR unreadable	Brightness/contrast on screen QR; shorten JSON vs URL-hosted map
flutter not found	Install Flutter SDK; reopen terminal; flutter doctor
No devices listed	USB debugging enabled; emulator started; adb devices
Silent STT path	Microphone granted; noisy room; articulate destination phrase
“Could not find place named...”	Spoken string must loosely match waypoint name field
Steps frozen at zero	Real device sensors; emulator limitations
scene_classifier/YOLO not active	Expect limited confirmation without matching .tflite in assets/models/ + rebuilt app

Detailed table: `docs/DEMO_AND_TESTING.md` §6.

13. From Development Build Toward Stores (Brief)

Publication is **outside** this handbook's procedural scope but the release path uses standard Flutter signing:

Platform	Outline
Android Play	Create your release keystore . Copy flutter_app/android/key.properties.example → flutter_app/android/key.properties and fill in real passwords and paths (never commit those secrets). Then run flutter build appbundle — see https://docs.flutter.dev/deployment/android .
iOS	Enroll in the Apple Developer Program , set up Xcode signing profiles, then run flutter build ipa — see https://docs.flutter.dev/deployment/ios .

Official references:

- Android: <https://docs.flutter.dev/deployment/android>
- iOS: <https://docs.flutter.dev/deployment/ios>

High-level bullets also appear in repo root **README.md**.

14. Project Team — UT Dallas (EPICS Computer Science)

- Amulya Prasad Rayabhagi (CS)
- Shresta Munikuntla (CS)
- Sadwitha Thopucharla (CS)
- Rushi Bikki (CS)
- Diep Doan (CS)
- Nandan Pabolu (CS)

Partner institutions and extended collaborators coordinate through EPICS stakeholder channels alongside **BVRIT** engineering teams when joint deployments arise.

15. License

Software is distributed under the **MIT License** — **LICENSE** at repository root:
https://github.com/nandanpabolu/GuidePoint_NAN/blob/main/LICENSE

Appendix A — Canonical Links Inside the Repository

Topic	Relative path after clone
Product & engineering README	README.md
Flutter subproject notes	flutter_app/README.md
Demo & feature tests	docs/DEMO_AND_TESTING.md
First Flutter install	docs/SETUP_FLUTTER_AND_RUN.md
Android emulator setup	docs/ANDROID_EMULATOR_SETUP.md

Topic	Relative path after clone
Map schema	docs/MAP_DATA_SCHEMA.md
Repository home: https://github.com/nandanpabolu/GuidePoint_NAN	

End of handbook.